MetroInsights Phase 4 - Interview Question Bank with Answers

Repository revision: `4405d167fc323f33be76e2c7f6614fd21cc574a8` (`main`), audited 2026-10-14. **Use this guide to explain the current repository, not an imagined production system.** The demand data is synthetic; operations inputs are configurable assumptions; travel times are graph estimates. Do not claim observed-data accuracy, live ridership, production dispatch, or personal experience that you did not have.

For each question, use the short answer to start, then the interview-ready answer. Source paths and line ranges are evidence anchors; interviewers may open those files and ask you to reason through them.

1. Project overview and architecture

Q1. Tell me about MetroInsights.

Short answer: MetroInsights is a Flask application for exploring synthetic Delhi Metro demand forecasts and rule-based service-planning scenarios.

Strong answer: A user selects a station, date, time and optionally a line. The browser posts those values to a Flask JSON API. The backend loads a saved scikit-learn model to estimate hourly demand, applies calendar and operating-window context, splits interchange demand using estimated line shares, calculates a headway/capacity scenario, and estimates network travel times with a weighted graph. The page updates KPIs, charts and tables. It is a planning/demo prototype; its forecast is trained on generated scenarios rather than observed passenger counts.

Evidence: `app/routes.py:235-398`; `app/forecasting.py:56-188`; `app/scheduling_optimizer.py:81-306`; `app/travel_graph.py:88-167`.

Likely follow-up: Is it real-time?

Follow-up answer: The UI requests a calculation immediately when controls change, but the repository does not connect to live AFC counts, train positions, or a live timetable. "On-demand scenario estimate" is accurate; "live ridership prediction" is not.

Common mistake: Calling it a live DMRC operations or dispatch system.

Q2. What problem does the project address, and who is it for?

Short answer: It demonstrates how station/time demand estimates could be connected to service-planning scenarios in one interface.

Strong answer: The implemented workflow lets a reviewer compare modeled demand by station/date/time, inspect a rule-based frequency/capacity recommendation, and explore estimated network travel times. The intended user is inferred from the interface and repository; there is no login, role model or operational workflow proving a specific DMRC employee audience. I would present it as an exploratory prototype until data and operating assumptions are validated.

Evidence: dashboard controls in `app/templates/index.html`; forecast endpoint and response in `app/routes.py:235-398`; data limitations in `DATA_AUDIT.md`.

Likely follow-up: Why not say passengers are the users?

Follow-up answer: The UI exposes station/date scenario analysis and destination times, but does not include passenger accounts, trip planning input, ticketing, accessibility routing or live service alerts. A passenger-facing product would need those product requirements.

Common mistake: Treating an inferred audience as a verified product requirement.

Q3. Walk me through the architecture.

Short answer: Jinja serves pages, browser JavaScript calls Flask, Flask orchestrates forecast/scheduling/graph modules, and CSV/JSON/joblib files supply data and model artifacts.

Strong answer: `run.py` imports the Flask app; `app/__init__.py` constructs it and imports routes to register decorators. Runtime modules load station data, operations and model artifacts. The browser sends a JSON request to `/api/predict`; the route parses the scenario and invokes calendar, forecasting, schedule and graph logic, then returns structured JSON. `dashboard.js` maps response fields into DOM elements and Chart.js charts. Separate numbered scripts prepare data and train artifacts offline.

Evidence: `run.py`; `app/__init__.py`; `app/routes.py`; `Procfile`; `app/static/js/dashboard.js:154-313`; `pipeline/01...`, `02...`, `03...`.

Likely follow-up: Is it a microservice architecture?

Follow-up answer: No. It is one Flask application with modules organized by responsibility, plus offline pipeline scripts. There are no separately deployed forecast or routing services in the repository.

Common mistake: Calling Python modules separate services.

Q4. Describe the end-to-end data flow.

Short answer: Station metadata is cleaned, synthetic examples are generated, a model is trained offline, and the web process uses the saved model for inference.

Strong answer: Pipeline 01 reads the raw station CSV and writes cleaned station and operational scenario files. Pipeline 02 creates typologies, interchange-share metadata and the large synthetic footfall CSV. Pipeline 03 prepares the features, uses a chronological holdout, trains/evaluates a regressor and saves joblib artifacts plus metrics. At web startup, forecasting loads those artifacts; per request it builds a feature row, predicts demand, then the route assembles schedules and travel times.

Evidence: `pipeline/01_audit_and_clean_data.py`; `pipeline/02_generate_calibrated_dataset.py`; `pipeline/03_train_and_evaluate_forecaster.py`; `app/forecasting.py:15-32`.

Likely follow-up: Does Render train the model during deployment?

Follow-up answer: No training command is in the `Procfile`; it starts `gunicorn run:app`. Training is a separate offline pipeline. The deployed process loads committed model artifacts.

Common mistake: Saying app startup retrains or refreshes the model.

Q5. What happens when a user changes the dashboard time?

Short answer: The browser sends a new JSON request and ignores an older response if a newer request has already started.

Strong answer: `initControls` listens to time and slider changes. `fetchPrediction` increments a request ID, reads station/date/time/line values, posts JSON, awaits response parsing and compares the captured ID with the current ID before rendering. A successful response updates KPIs, charts, line allocations and travel destinations. Old calls are not aborted, but stale results cannot overwrite a newer response.

Evidence: `app/static/js/dashboard.js:41-201` and `203-544`.

Likely follow-up: Does this prevent wasted server work?

Follow-up answer: No. It prevents stale rendering only. To save work, the client could use `AbortController`, debounce slider events, or the server could cache equivalent requests.

Common mistake: Calling the request ID a cancellation mechanism.

2. Data, features and model

Q6. What data trained the forecaster?

Short answer: It was trained on synthetic station/hour scenarios generated by this project, not observed station ridership.

Strong answer: Pipeline 02 generates 3,153,696 records from 228 stations, 728 dates and 19 hourly slots per date. It uses authored station weights, line weights, day/hour/month factors, holiday factors and seeded random sampling. This supports reproducible simulation and UI experimentation, but a model trained and tested on those generated targets has not been validated against passenger counts.

Evidence: `pipeline/02_generate_calibrated_dataset.py`; `models/evaluation_metrics.json`; `DATA_AUDIT.md`.

Likely follow-up: Why is synthetic data useful at all?

Follow-up answer: It can exercise the end-to-end feature, model, API and visualization workflow when observed data is unavailable. Its value is software and scenario demonstration; it cannot establish real-world accuracy or validate the assumptions used to generate it.

Common mistake: Saying "3.15 million records" as if they were observed measurements.

Q7. Which model and features does the project use?

Short answer: The training pipeline fits a `HistGradientBoostingRegressor` to 23 time, calendar, station, line and historical aggregate features; it also compares baselines.

Strong answer: Features include hour/day/month and day-of-year values, cyclical sine/cosine encodings, weekend/holiday/workday flags, encoded station/line/typology, interchange and line-count fields, and historical station/hour-day/month means. Runtime constructs a dictionary and selects columns in saved `feature_cols` order before calling `.predict()`. Training and runtime metadata must therefore remain aligned.

Evidence: `pipeline/03_train_and_evaluate_forecaster.py:48-196`; `app/forecasting.py:56-116`; metadata producer/consumer in that pipeline and `app/forecasting.py`.

Likely follow-up: Why use sine/cosine encodings?

Follow-up answer: They represent cyclic proximity: 23:00 is close to 00:00 in time, even though their integer values are far apart. The paired sine and cosine locate a point on a cycle. The model still has additional integer/calendar features.

Common mistake: Claiming that cyclical encoding alone captures every seasonal pattern.

Q8. How did you prevent temporal leakage?

Short answer: The training preparation computes historical aggregate features using prior dates for training rows, then uses training-period aggregates for held-out rows.

Strong answer: The important principle is that a row's target must not contribute to the historical feature used to predict that same row. `prepare_annual_data` sorts and creates prior-only aggregates for training history, while the chronological split holds dates starting 2026-10-01 out. For held-out rows, learned aggregates are based on the earlier training period. I would still test this with a small hand-computable dataset and verify each aggregate's source dates.

Evidence: `pipeline/03_train_and_evaluate_forecaster.py:48-196`, especially the history aggregate and split logic.

Likely follow-up: Is a time split enough to prove leakage-free evaluation?

Follow-up answer: No. The split is necessary but not sufficient. Feature construction must also exclude future/test information, preprocessing must be fit only on training data, and repeated stations across time must be handled intentionally. Even a correct synthetic time split only evaluates synthetic targets.

Common mistake: Equating chronological splitting alone with complete leakage prevention.

Q9. What do the reported model metrics mean?

Short answer: The tracked synthetic holdout reports MAE 27.90, RMSE 54.82, R-squared 0.9947 and MAPE 5.16%; these score fit to generated data, not real riders.

Strong answer: MAE is the mean absolute passenger-count error, so it is interpretable in target units. RMSE penalizes larger errors more. R-squared compares residual variation to target variation on that test set. MAPE is calculated only on nonzero target rows because division by zero is undefined. The test period is later than training, but both train and test samples come from the same synthetic generator family, so the score can reward learning those assumptions.

Evidence: `models/evaluation_metrics.json`; `pipeline/03_train_and_evaluate_forecaster.py:197-295`; `app/forecasting.py:191-198`.

Likely follow-up: What evidence would establish real-world accuracy?

Follow-up answer: Time-stamped observed station-level targets with documented provenance, a leakage-safe chronological evaluation against simple baselines, error analysis by station/hour/day type, and ideally prospective validation after deployment. Current repository data cannot provide this evidence.

Common mistake: Presenting the R-squared as proof the model is 99.47% accurate on DMRC passengers.

Q10. Why exclude zero targets from MAPE?

Short answer: Percentage error divides by the actual value, so zero targets make MAPE undefined.

Strong answer: For nonzero actual value `y`, absolute percentage error is `abs(y - y_hat) / abs(y)`. At zero, division is undefined and tiny actual values can make the percentage disproportionately large. The pipeline calculates MAPE only for positive/nonzero targets, so the metric describes that subset and should be labeled accordingly. MAE/RMSE remain defined at zero.

Evidence: `pipeline/03_train_and_evaluate_forecaster.py:197-210`.

Likely follow-up: What alternative would you report?

Follow-up answer: MAE and RMSE are already reported. Depending on evaluation goals, one could add WAPE or a scale-aware metric, but the choice should be justified for sparse demand and evaluated on held-out observed counts.

Common mistake: Saying MAPE always handles zero values safely.

Q11. How does hourly prediction become a minute-level estimate?

Short answer: It linearly interpolates between two hourly model predictions using the selected minute fraction.

Strong answer: The inference function calculates `v0` at the selected hour and `v1` at the next hour, then computes `v0 + (v1-v0) * minute/60`, rounds, and clamps at zero. For example, minute 15 weights the next hour by 0.25. This creates a smooth estimate from hourly anchors; the ML model itself does not predict minute-by-minute demand.

Evidence: `app/forecasting.py:119-130`.

Likely follow-up: What happens at 23:00?

Follow-up answer: The next anchor is capped at hour 23, so `v1` equals the 23:00 prediction. The modeled service gate closes at 23:30, and selected times at/after that boundary return zero from this function.

Common mistake: Calling this minute-resolution training or inference.

Q12. How are date effects used?

Short answer: The project derives weekday, workday, holiday/occasion and cyclical calendar features from the selected date.

Strong answer: `get_calendar_context` classifies day type and known listed occasions; runtime feature construction converts parts into booleans/categories and cyclical encodings. It is a project-maintained calendar, not a live calendar API. An unlisted event or unconfirmed closure may not be reflected. The `is_workday` parameter passed into scheduling can override weekday-only peak inference.

Evidence: `app/calendar_context.py:26-111`; `app/forecasting.py:66-109`; `app/scheduling_optimizer.py:38-49`.

Likely follow-up: Is a weekday always treated as a workday?

Follow-up answer: Calendar logic can mark holidays/non-workdays; the scheduler accepts the resulting `is_workday` override. If that argument is omitted, scheduler fallback uses weekday index only.

Common mistake: Describing the occasion list as a continuously updated official calendar feed.

3. Scheduling and capacity

Q13. What does capacity utilization mean here?

Short answer: It is modeled station boarding demand divided by modeled hourly boarding-clearance capacity, not observed onboard occupancy.

Strong answer: Per-train clearance is estimated as comfort capacity times one minus an assumed through-load factor: 65% at peak and 40% off-peak, with a minimum floor of 250. Hourly clearance is that amount multiplied by trains/hour. Utilization divides station/line demand by this hourly clearance. The calculation does not know who is already on the train, how many alight, or which train reaches the station next.

Evidence: `app/scheduling_optimizer.py:119-160`; operations assumptions in `data/cleaned/metro_line_operations.csv`.

Likely follow-up: How should a more realistic occupancy metric work?

Follow-up answer: Track a train's onboard load segment by segment: previous onboard load minus alighters plus boarders at each stop, bounded by train capacity. That needs origin-destination or station boarding/alighting counts, train assignments and service patterns. The current repository lacks these inputs.

Common mistake: Equating the displayed percentage with measured train crowding.

Q14. How does the service recommendation choose a headway?

Short answer: It compares demand with baseline clearance capacity, selects maintain/increase/decrease behavior, clamps frequency to configured constraints, then rounds to a practical headway list.

Strong answer: The optimizer loads comfort/nominal capacity, signaling minimum, policy maximum, fleet and turnaround from a CSV. It calculates a fleet and signaling maximum frequency and a policy minimum frequency. It picks a baseline headway from

peak/off-peak/taper rules, estimates boarding clearance, compares projected demand, clamps a target frequency and converts it to a headway. It rounds that headway to the closest standard value and recomputes metrics. It is deterministic rules over assumptions, not a learned optimizer or timetable solver.

Evidence: `app/scheduling_optimizer.py:81-249`; `data/cleaned/metro_line_operations.csv`.

Likely follow-up: Is that schedule operationally feasible?

Follow-up answer: Not established. It uses configured values but does not model trip-level timetables, depot allocation, train positions, platform dwell, interlining or network-wide conflicts. Source audit also found practical headway rounding can make reported frequency exceed the earlier fleet clamp.

Common mistake: Saying the app creates a deployable train timetable.

Q15. What is the headway/fleet constraint issue?

Short answer: Frequency is clamped before headway rounding; choosing the nearest practical headway can raise frequency past the clamp.

Strong answer: For the tracked Yellow Line example, a 25 trains/hour fleet clamp implies a 2.4-minute raw headway. Standard options 2.3 and 2.5 are tied at 0.1 minutes; list order chooses 2.3, which corresponds to about 26.1 trains/hour. Thus the final recommendation can exceed the calculated fleet maximum. The correct repair is to select a practical headway that preserves all constraints or reapply constraints after rounding, then add invariant tests.

Evidence: `app/scheduling_optimizer.py:99-104`, `145-151`; `STANDARD_HEADWAYS` at lines 31-35; Yellow Line CSV row.

Likely follow-up: What invariant would you test?

Follow-up answer: Final `60 / recommended_headway` must be no greater than `min(floor(fleet*60/turnaround), floor(60/min_signal_headway))`, while the final headway remains within signaling and policy bounds. Tests should include rounding ties and bounds that leave no standard interval.

Common mistake: Assuming that because a value was clamped, later rounding cannot invalidate the constraint.

Q16. How is demand allocated at an interchange?

Short answer: The total forecast is multiplied by stored estimated proportions for each line, with equal-share fallback.

Strong answer: `get_interchange_line_split` loads `proportions` for a station from JSON. `optimize_station_schedule` multiplies the total by each line's share, rounds, and calls the line optimizer. The shares are estimated/generated assumptions rather than observed transfer or line boarding measurements. It does not model OD passengers transferring between lines.

Evidence: `app/scheduling_optimizer.py:52-64`, `252-306`; `data/synthetic/line_distribution_model.json`.

Likely follow-up: Do shares always sum to one?

Follow-up answer: The function returns stored proportions without validating or renormalizing them. Equal fallback is rounded to four decimals. Input validation and a residual correction would be appropriate improvements.

Common mistake: Calling the line proportions observed passenger shares.

Q17. Why call the schedule optimizer rule-based?

Short answer: Its decisions come from if/else thresholds and arithmetic over configuration, not from fitting or solving a train schedule optimization model.

Strong answer: It calculates required frequency from demand and per-train clearance, compares this with baseline conditions, applies explicit peak/light-demand action rules, bounds frequency and selects a practical headway. There is no objective function minimized over a timetable, no constraints solver, and no live state feedback loop.

Evidence: `app/scheduling_optimizer.py:129-151`.

Likely follow-up: What would a real optimization formulation need?

Follow-up answer: A defined objective (waiting, crowding, energy, operating cost), time-dependent demand and OD flows, fleet/turnaround/depot constraints, signaling separation, dwell times, infrastructure capacity, service policies and validated operational data. Then solve and validate against schedule feasibility.

Common mistake: Using "AI optimizer" as a synonym for threshold rules.

4. Network graph and algorithms

Q18. How does the travel graph represent a station on multiple lines?

Short answer: Nodes are `(station, line)` tuples, so changing lines happens over a separate weighted transfer edge.

Strong answer: The topology provides ordered stop sequences; the graph builder links only adjacent names within a line and inserts bidirectional interchange edges with a transfer penalty. A station served by Blue and Yellow therefore has two graph nodes. Dijkstra pays the transfer edge weight only when a path changes from one line node to another.

Evidence: `app/network_topology.py:21-169`; `app/travel_graph.py:88-136`.

Likely follow-up: Why not use station names as nodes?

Follow-up answer: One station-name node would collapse platforms/lines and could allow switching lines for free. Line-expanded nodes preserve the state needed to charge a transfer.

Common mistake: Assuming the transfer edge is a physical track segment.

Q19. Explain Dijkstra's algorithm as used here.

Short answer: It repeatedly expands the least-known travel-time node and relaxes neighboring edges using a min-heap.

Strong answer: The function initializes every line-node at the origin station with distance zero, heapifies those sources, pops the smallest candidate, skips already visited nodes, and checks each neighbor. If `d + w` improves that neighbor's distance, it records and pushes the new candidate. It then collapses line nodes to the best time per station and removes the origin. Nonnegative edge weights are required.

Evidence: `app/travel_graph.py:139-167`.

Likely follow-up: Why multiple starting nodes?

Follow-up answer: The origin may be an interchange. Starting from each of its line nodes at zero means a journey that begins at the station does not pay an artificial transfer penalty just to choose the initial line.

Common mistake: Describing it as exhaustive enumeration of every possible route.

Q20. Are the travel times actual journey times?

Short answer: No. They are shortest-path estimates over topology edges weighted from coordinates or a fallback and a fixed average speed.

Strong answer: `build_graph` maps station labels, estimates adjacent-station distance with haversine distance divided by 32 km/h, enforces a one-minute minimum, and uses a 2.5-minute fallback for missing/repeated/implausible coordinates. Transfers have a constant five-minute weight. This omits track geometry, scheduled dwell, departure time, actual transfer-walk distances and service disruptions.

Evidence: `app/travel_graph.py:24-26`, `61-67`, `88-136`; `app/network_topology.py:147-155`.

Likely follow-up: How would you improve it?

Follow-up answer: Use a sourced, licensed current GTFS/static timetable or verified segment travel times, model service calendars and transfer walking times, version the source and test route coverage/known journeys. Live arrival estimates would need a real-time feed.

Common mistake: Calling straight-line coordinate distance "track distance."

5. API, web stack and frontend

Q21. What does `/api/predict` accept and return?

Short answer: It accepts station/date/time/optional line from JSON or query args and returns forecast, schedule, calendar, profile and travel-time data as JSON.

Strong answer: It defaults station/time when absent, parses the date/time, calculates calendar/service state, obtains station metadata, performs forecast/profile/schedule logic, optionally chooses a line-level headline or aggregate interchange headline, calculates travel-time entries, and includes provenance labels. Invalid date/time `ValueError` returns a 400 JSON error. There is not comprehensive request-schema validation for unknown station or line values.

Evidence: `app/routes.py:235-398`.

Likely follow-up: What happens for an unknown line filter?

Follow-up answer: The matching loop does not find a line, and the route retains its default recommendation rather than returning a validation error. That can make the output scope unexpected; validate requested line against `all_lines`.

Common mistake: Saying every invalid field is rejected explicitly.

Q22. Why Flask?

Short answer: Flask provides a small WSGI application and Jinja rendering suitable for this single-app prototype.

Strong answer: The project needs a few page routes and JSON endpoints around Python data/ML modules. Flask's route decorators and template rendering keep the web layer relatively small. Django could be preferable for built-in admin/auth and ORM needs; FastAPI could be preferable for typed API schemas or asynchronous API needs. No database or async workflow currently requires those features.

Evidence: `requirements.txt`; `app/__init__.py`; `app/routes.py`; `Procfile`.

Likely follow-up: Why not add FastAPI now?

Follow-up answer: A rewrite would not solve the main limitations, which are data validity, request validation, tests and operational model design. I would keep Flask unless requirements justify a migration.

Common mistake: Claiming Flask was benchmarked as the fastest option.

Q23. Is there a database?

Short answer: No database is tracked or used in the current application flow; the app reads CSV/JSON files and model artifacts.

Strong answer: Station and operations configuration are file-based, and the trained model/metadata are joblib artifacts. There is no ORM, schema, migration, or database configuration. A relational database could help manage versioned stations, operations scenarios, users and queryable history; it would not substitute for a verified passenger data source.

Evidence: `app/routes.py`, `app/forecasting.py`, `app/scheduling_optimizer.py`, repository file inventory.

Likely follow-up: Is the app state persistent?

Follow-up answer: The selected scenario is sent per request and model/data files are read from the deployed filesystem. The app does not persist user sessions or forecast history in a database. Platform filesystem persistence is a separate deployment concern for any future writes.

Common mistake: Inventing a database layer because CSV files look tabular.

Q24. How does the frontend update charts safely against race conditions?

Short answer: It uses an incrementing request ID to suppress stale response rendering and destroys old Chart.js instances before replacing them.

Strong answer: Request sequencing protects UI consistency, while `demandChart.destroy()` and `lineChart.destroy()` prevent stacked chart instances when new responses arrive. These solve different issues. The browser still sends all requests, and dynamic row construction uses some `innerHTML` interpolation, which should be replaced by text nodes/validated styles if values could become untrusted.

Evidence: `app/static/js/dashboard.js:154-201`, `315-405`, `407-489`, `498-544`.

Likely follow-up: Is using `textContent` elsewhere sufficient to eliminate XSS?

Follow-up answer: No. Security depends on every sink. The table renderers interpolate dynamic destination/line/query values into `innerHTML`; those specific paths need safe DOM construction or strict sanitization.

Common mistake: Assuming server-generated data is automatically safe forever.

6. Reliability, performance, security and testing

Q25. What can prevent the deployed app from starting?

Short answer: Required model/data reads happen at module import, so missing files or incompatible/corrupt artifacts can fail worker startup.

Strong answer: Importing `run:app` imports the Flask package and routes; those import forecasting/scheduling code that loads joblib artifacts and CSV/JSON data into globals. Gunicorn workers repeat that initialization. The deployment must include compatible dependencies and required compact runtime files/artifacts. The generated large training CSV is not needed for ordinary inference if the saved artifacts exist, but is needed by the training pipeline unless regenerated.

Evidence: `Procfile`; `run.py`; `app/__init__.py`; `app/forecasting.py:15-32`; `app/scheduling_optimizer.py:19-29`.

Likely follow-up: How would you improve startup reliability?

Follow-up answer: Validate required files and schemas at startup with clear diagnostics, add a deployment smoke check, pin compatible artifact/library versions, make artifact loading explicit, and avoid silently continuing with partial data unless a defined fallback is safe.

Common mistake: Saying `.gitignore` deletes or regenerates ignored CSV files.

Q26. How would you deploy it on Render?

Short answer: Connect the Git repository, configure Python build dependencies, and use the tracked Gunicorn `Procfile` command with model/runtime artifacts included.

Strong answer: `Procfile` specifies `web: gunicorn run:app`. Render needs the repository root, dependency installation from `requirements.txt`, and required model/joblib, station, operation and line-share files. The large generated training CSV is ignored and is not read by the serving path. If retraining on Render is required, provide the data or explicitly regenerate it in a suitable persistent/build job; do not assume it appears because its source is in Git. Free instance sleep affects latency, not model math.

Evidence: `Procfile`, `.gitignore`, `requirements.txt`, runtime file loads and pipeline 03 input paths.

Likely follow-up: Could I deploy without the 130 MB CSV?

Follow-up answer: For serving the committed model, yes in the current code path, provided all runtime artifacts/config are included. For rerunning pipeline 03, no; its synthetic CSV input must exist or be regenerated. Build/runtime storage and memory limits must also be considered.

Common mistake: Claiming ignored training data is fetched automatically from GitHub or Render.

Q27. What are the main performance risks?

Short answer: Large offline generation/training can be memory- and CPU-intensive; online inference is comparatively small, but startup loads artifacts and each request computes a graph search.

Strong answer: Pipeline 02 writes roughly 3.15 million hourly records; pipeline 03 loads and transforms them. These should be run in a suitable offline environment. Runtime model inference builds 19 predictions for a full profile and may run multiple model calls per request; travel Dijkstra runs over the in-memory network graph. With only a few hundred stations this graph is modest, but repeated requests could benefit from caching graph and static response components. The browser's slider can issue many uncancelled requests.

Evidence: `pipeline/02_generate_calibrated_dataset.py:239-331`; `pipeline/03...`; `app/forecasting.py:133-188`; `app/routes.py:347-358`; `dashboard.js:61-66`.

Likely follow-up: What would you profile first?

Follow-up answer: Measure request latency by stage (feature/model, profile loop, scheduler, graph, serialization), worker memory after artifact load, and pipeline peak memory/runtime. Optimize based on measurements; likely candidates are request deduplication/caching and streaming/chunked offline data processing.

Common mistake: Guessing a bottleneck without timing it.

Q28. What security issues would you address?

Short answer: Validate request inputs, replace dynamic `innerHTML` render paths, and treat joblib artifacts as trusted-only inputs.

Strong answer: The browser uses `textContent` for many fields, but dashboard line and travel rows interpolate dynamic values into `innerHTML`; this is an injection sink. Server routes trim some input and catch date parsing errors, but do not comprehensively validate station/line membership. Joblib artifacts should never come from user uploads or untrusted sources because deserialization can execute code. Add request schemas, safe DOM creation, artifact provenance and security regression tests.

Evidence: `app/static/js/dashboard.js:407-451`, `498-544`; `app/routes.py:235-248`; `app/forecasting.py:22-23`.

Likely follow-up: Does Flask autoescape every rendered value?

Follow-up answer: Jinja autoescaping generally applies to HTML template variable output, but it does not protect JavaScript DOM `innerHTML` assignments or unsafe `safe` filters. Each output context needs appropriate encoding.

Common mistake: Treating autoescaped server templates as protection for client-side rendering.

Q29. What testing exists?

Short answer: No automated test suite or test configuration is tracked in the repository.

Strong answer: I would add focused unit tests for date/service boundaries, feature-column order, unknown station behavior, headway constraints after rounding, line shares and Dijkstra relaxation. Pipeline tests should use a tiny fixture and verify deterministic output/schema. Flask tests should cover valid and invalid JSON, closed service, unknown station/line and artifact failures. A browser test should verify a mocked response maps to the expected KPIs and that stale responses cannot overwrite newer state.

Evidence: tracked file inventory contains no test directory or test config; relevant functions: `app/operating_hours.py`, `forecasting.py`, `scheduling_optimizer.py`, `travel_graph.py`, `routes.py`, `dashboard.js`.

Likely follow-up: Why not just test the happy path?

Follow-up answer: Boundary and invalid paths contain important behavior here: 23:30 uses an exclusive end, unknown line silently falls back, empty line lists can make `max` fail, and practical rounding can exceed the fleet cap.

Common mistake: Saying tests pass when no tests were run or exist.

Q30. What does the closed-hours gate do?

Short answer: It models service only from 05:00 inclusive to 23:30 exclusive and zeros the selected-time demand outside that interval.

Strong answer: `is_service_operating` converts hour/minute to minutes after midnight and checks `start <= minute_of_day < end`. Thus 23:29 is open and exactly 23:30 is closed. The API skips selected-time minute/hour prediction when closed and overrides schedule headline values, but still builds the daily profile. This is a configured modeled window, not an official live service calendar.

Evidence: `app/operating_hours.py:3-10`; `app/routes.py:249-257`, `326-345`.

Likely follow-up: Is the 23:00 chart point a full operating hour?

Follow-up answer: It is an hourly model bucket generated as part of the 05:00-23:00 profile. It does not represent precise passenger activity through the 23:30 boundary.

Common mistake: Treating the end time as inclusive or assuming a closed selection skips every model call.

7. Alternative designs and very difficult questions

Q31. Why not deploy the 130 MB CSV and retrain on every startup?

Short answer: It is a training input, not needed for ordinary inference with the saved model, and retraining at web startup would increase boot time/resources and make deployments less predictable.

Strong answer: The app loads `footfall_forecaster.joblib` and metadata. Pipeline 03 consumes the generated CSV to create new artifacts. Separating offline training from online inference keeps web startup bounded and makes the deployed artifact reproducible. If data/model retraining is needed, run a versioned pipeline job and publish compatible artifacts deliberately. Retain reproducible data in object storage or an approved data store if Git size is unsuitable.

Evidence: `app/forecasting.py:15-32`; `pipeline/03_train_and_evaluate_forecaster.py:38-43`; `.gitignore`.

Likely follow-up: How do you know the model matches the deployed metadata?

Follow-up answer: They are separate joblib files loaded together; the repo does not show an explicit artifact manifest/hash compatibility check. Add a shared version/schema hash and validate `feature_cols` and library versions at load.

Common mistake: Thinking Git ignore makes the file available to Render.

Q32. What would you need to estimate actual passenger load at a selected station?

Short answer: Station-level boardings and alightings linked to a train's prior onboard load, plus train/service assignment and capacity data.

Strong answer: For each train and segment, update onboard load as previous load minus passengers alighting plus passengers boarding. Enforce physically valid bounds and carry that state to the next stop. A station/hour aggregate alone cannot tell how many are already onboard. Origin-destination or train-level AFC/automatic passenger count data and verified train schedules would support a better estimate. Current utilization is a demand-to-clearance ratio, not occupancy.

Evidence: `app/scheduling_optimizer.py:119-160`; missing input categories in `DATA_AUDIT.md`.

Likely follow-up: Would a station's total entries be enough?

Follow-up answer: No. It omits direction, train arrival timing, alighting, transfers and distribution across trains. At minimum, boarding/alighting by direction and time interval plus service frequency is needed; train-level load requires stronger linkage.

Common mistake: Dividing hourly station footfall by train capacity and calling it train occupancy.

Q33. Why are high synthetic holdout metrics not sufficient?

Short answer: The model is evaluated against targets generated by the same authored scenario process, not independent real passenger counts.

Strong answer: A flexible model can recover patterns built into the generator and score very well on held-out dates drawn from that generator. This tests pipeline consistency/generalization within those synthetic rules. It cannot tell whether the generator's station weights, rush-hour curves or holiday factors resemble actual DMRC behavior. External validation requires observed, independently sourced, chronological ridership data.

Evidence: `pipeline/02_generate_calibrated_dataset.py`; `pipeline/03_train_and_evaluate_forecaster.py`; `DATA_AUDIT.md`; `models/evaluation_metrics.json`.

Likely follow-up: Is a synthetic holdout useless?

Follow-up answer: No. It can detect code regressions and compare models under the generator's assumptions. The claim must be scoped correctly and supplemented with real-data evaluation before operational use.

Common mistake: Treating test split as independent ground truth when both subsets share synthetic origin.

Q34. What happens if the station is not in metadata?

Short answer: The station-info helper returns unknown metadata, but forecast encoding falls back to station index zero; the request is not consistently rejected.

Strong answer: `get_station_info` returns an "Unknown" line with empty line list and null coordinates. In forecasting, `station2idx.get(station, 0)` maps an unknown station to index zero and historical features fall back through metadata defaults. This can produce a plausible-looking but misleading forecast. The schedule path can also fail later when no lines produce recommendations. The API should validate station membership and return a clear 400/404 before model work.

Evidence: `app/forecasting.py:34-53`, `56-85`; `app/routes.py:252-253`; `app/scheduling_optimizer.py:259-306`.

Likely follow-up: Why is a default index dangerous?

Follow-up answer: It turns invalid input into another station's encoded feature rather than surfacing a validation error, causing silent semantic corruption.

Common mistake: Saying the request is safe because `get_station_info` has a fallback.

Q35. How do you calculate time complexity of the graph search?

Short answer: With adjacency lists and a binary heap, Dijkstra is typically `O((V+E) log V)` time and `O(V+E)` graph/search space.

Strong answer: Each node is finalized once; each outgoing edge can be considered during relaxation; successful improvements push heap candidates, which incur logarithmic heap operations. The exact heap can contain stale candidates, so a conservative bound is commonly expressed in terms of E pushes, yielding `O(E log E)` or `O((V+E) log V)` for a simple graph. Memory includes adjacency, distances, visited nodes and heap entries. This network is small enough that correctness and data quality are more important than asymptotic optimization.

Evidence: `app/travel_graph.py:139-167`.

Likely follow-up: What assumption makes Dijkstra correct?

Follow-up answer: All edge weights are nonnegative. The current graph builder uses positive or fallback travel minutes and positive transfer penalties.

Common mistake: Saying Dijkstra supports negative edge weights.

Q36. How do you handle model/data version compatibility?

Short answer: The current code loads joblib files directly; explicit artifact version/hash validation is a recommended improvement.

Strong answer: Training writes the estimator and metadata separately, and inference loads both at import time. The feature DataFrame is ordered by metadata `feature_cols`, which is one compatibility guard, but there is no complete manifest binding model, metadata, training data version and sklearn/Python versions. I would save a versioned artifact bundle/manifest, validate feature schema and dependency versions, and fail with an actionable startup message.

Evidence: `pipeline/03_train_and_evaluate_forecaster.py:211-295`; `app/forecasting.py:15-32`, `111-116`.

Likely follow-up: What is the security concern with joblib?

Follow-up answer: Joblib uses Python object serialization mechanisms; untrusted artifacts can execute code during loading. Only load trusted, integrity-checked model artifacts.

Common mistake: Treating a `.joblib` file as a harmless data-only format.

Q37. How would you evolve the app toward a real planning tool?

Short answer: Replace synthetic inputs with sourced ridership and operations feeds, establish validation, then improve load and timetable models incrementally.

Strong answer: First document sources, licenses, update cadence and schemas. Build immutable raw ingestion and data-quality checks for station/hour counts and train operations. Retrain with chronological and station-aware validation, report uncertainty and baselines, and compare predictions with observed periods. Then represent boardings/alightings and onboard train loads; replace generic headway rules with a constrained timetable model using verified fleet, signaling, depot and dwell inputs. Add tests, monitoring, versioned artifacts and access/security controls before operational use.

Evidence: gaps and recommendations in `DATA_AUDIT.md`; current modules show these assumptions are file-based and simplified.

Likely follow-up: What would you do first?

Follow-up answer: Secure an approved observed station/time ridership dataset and define a validation target. Without that, adding a more complex model risks making synthetic assumptions look more authoritative.

Common mistake: Starting with a more complex algorithm before fixing ground truth.

8. Debugging scenarios

Q38. A displayed KPI disagrees with the API response. How do you debug it?

Short answer: Trace the value from request payload through API JSON to the exact DOM renderer.

Strong answer: Inspect browser Network request values and response status/body. Check `scheduling.headline` and relevant `footfall` fields in JSON. Compare those with `renderDashboard` selection logic, including whether a line filter is active and whether service is closed. Finally inspect the DOM element and console errors. This separates input state, backend calculation, response contract, and rendering issues.

Evidence: `dashboard.js:154-313`; `routes.py:363-398`.

Likely follow-up: What if the response is correct but the chart is not?

Follow-up answer: Check chart data selection, chart destruction/recreation, Chart.js loaded state, array lengths/types and chart-specific caught console error.

Common mistake: Changing backend calculations before checking the actual response.

Q39. Travel destinations are missing. What would you inspect?

Short answer: Check station-name matching, topology adjacency/interchange definitions, graph source nodes and reachable-node results.

Strong answer: First confirm the canonical station string exists in the loaded cleaned CSV. Check `_normalize`/aliases and whether the topology node mapped. Then inspect `INTERCHANGES`, graph edges and coordinate fallback. `shortest_times_from` returns only graph-reachable stations and returns an empty mapping if no source node matches. Finally inspect route sorting/line labeling and frontend array handling.

Evidence: `app/travel_graph.py:29-49`, `70-85`, `88-167`; `app/network_topology.py`.

Likely follow-up: Could a station have valid coordinates but still not be reachable?

Follow-up answer: Yes. Coordinates do not create edges; the topology must place a station in an ordered line sequence or an interchange definition.

Common mistake: Assuming nearest geographic stations are automatically connected.

9. Resume cross-examination and truthful framing

Q40. You wrote "AI-based metro demand prediction." What exactly is AI here?

Short answer: A scikit-learn regression model estimates hourly demand from engineered features; the scheduling and routing portions are deterministic rules/graph algorithms.

Strong answer: Pipeline 03 trains a histogram-based gradient boosting regressor and compares it with baselines. Runtime prepares 23 features and calls the estimator. The demand targets are synthetic, so I would qualify the project as a machine-learning simulation/prototype. Headway recommendations use hand-coded conditions and configuration; destination times use Dijkstra, not ML.

Evidence: `pipeline/03_train_and_evaluate_forecaster.py`; `app/forecasting.py`; `app/scheduling_optimizer.py`; `app/travel_graph.py`.

Likely follow-up: What was the real-world performance?

Follow-up answer: The repository cannot establish real-world performance because it contains no observed station-level ridership target. I can report the synthetic holdout metrics with that caveat, but not represent them as DMRC accuracy.

Common mistake: Saying every subsystem is AI-powered.

Q41. Your resume says the app supports "real-time user input." Is the data real-time?

Short answer: Input changes trigger an immediate recalculation, but the data and service state are not live feeds.

Strong answer: The browser posts user-selected station/date/time and displays the response without a full page reload. The model and operation parameters are loaded from repository artifacts/configuration. There is no live ridership or train feed. I would word it as "interactive, on-demand scenario analysis" to avoid confusion.

Evidence: `app/static/js/dashboard.js:154-201`; runtime file imports and `DATA_AUDIT.md`.

Likely follow-up: What does "real time" mean to you here?

Follow-up answer: Only that the UI responds after a user interaction; it does not mean streaming or live operational data.

Common mistake: Letting a UI interaction claim imply a live data pipeline.

Q42. Which numbers can you safely put on a resume?

Short answer: Cite measured repository facts, and label generated data and synthetic evaluation honestly.

Strong answer: Defensible numbers include 228 station records in the audited source directory, 3,153,696 generated hourly scenarios, 23 model features, and the exact synthetic train/test counts and metrics if useful. Be explicit that these are repository/simulation counts. Do not claim 226 live operational stations, observed ridership coverage, real-world accuracy, or user impact unless independently established.

Evidence: `DATA_AUDIT.md`; `pipeline/02_generate_calibrated_dataset.py`; `models/evaluation_metrics.json`.

Likely follow-up: Did the project reduce crowding?

Follow-up answer: There is no deployed operational intervention or before/after outcome data in this repository, so I cannot claim a measured crowding reduction. The app demonstrates scenario estimates and recommendations.

Common mistake: Turning generated row count into an impact metric.

Q43. Tell me about a difficult technical issue you solved.

Short answer: Use a real issue you personally investigated; the repository can support a technical example about detecting model/operational interpretation gaps, but do not claim work you did not do.

Strong answer template: "I noticed **[actual observed symptom]**. I traced it through **[files/functions you personally inspected]**, isolated **[verified cause]**, and changed **[your actual change]**. I checked the result with **[test or evidence you actually used]**. The lesson was **[specific takeaway]**." If you personally did not implement a fix, say you identified the risk and explain the proposed fix rather than taking credit for implementation.

Evidence: The repository contains examples an interviewer may explore: exclusive 23:30 service boundary (`app/operating_hours.py`), station fallback (`app/forecasting.py`), and final headway rounding versus fleet limit (`app/scheduling_optimizer.py`). These are technical evidence, not proof of who authored or fixed them.

Likely follow-up: What would you do differently?

Follow-up answer: Give a specific improvement grounded in the actual episode, such as adding a regression test before refactoring or validating the full constraint after rounding. Do not invent team conflict, outage impact or user feedback.

Common mistake: Presenting a repository issue as a personal achievement without knowing who found/fixed it.

Q44. What would you say if asked about a weakness in this project?

Short answer: The biggest limitation is lack of observed ridership and independently verified operating data.

Strong answer: The architecture demonstrates a useful end-to-end workflow, but the synthetic targets cannot validate real demand. The current utilization is a boarding-clearance scenario, not onboard occupancy; schedule rules omit train-level feasibility; travel times use simplified weights. I would prioritize real data provenance and validation, then tests and constraints. This is a clear scope limitation, not something to hide behind high synthetic metrics.

Evidence: `DATA_AUDIT.md`; `app/scheduling_optimizer.py`; `app/travel_graph.py`.

Likely follow-up: Then why build it?

Follow-up answer: It demonstrates data preparation, feature engineering, model training/serving, a Flask API, interactive visualization, a rule-based scenario engine and graph search. It is a prototype whose assumptions and gaps are explicit.

Common mistake: Claiming limitations do not matter because this is a portfolio project.

10. Behavioral question practice without invented experience

Q45. Tell me about a time you worked with ambiguous requirements.

Short answer: Answer with a real example and show how you converted ambiguity into explicit assumptions and a decision.

Strong answer framework: State the actual unclear requirement; identify who needed the result; describe the question/assumption you documented; explain how you validated it; say what changed. For this project, only if true, you could discuss clarifying that capacity utilization should represent boarding clearance rather than measured onboard load. Do not claim a conversation or stakeholder approval unless it happened.

Evidence: Code/data can ground the distinction (`scheduling_optimizer.py:119-160`), but not prove a personal conversation.

Likely follow-up: What if the assumption was wrong?

Follow-up answer: Explain how you would version the assumption, expose it in output, compare alternatives and update the calculation when better evidence arrives.

Common mistake: Inventing a stakeholder or a conflict to fill out STAR.

Q46. How do you respond when a reviewer finds a defect?

Short answer: Reproduce it, isolate the responsible contract or invariant, make a focused change, and verify the boundary case.

Strong answer framework: Describe one real review comment; show the evidence you inspected; acknowledge what was wrong; explain the test or check added; state what you learned. If you have not yet experienced this, say how you would handle it rather than narrating a fictional event.

Evidence: Useful candidate test boundaries in the current repo include exactly 23:30, malformed dates, unknown station/line, headway rounding ties, and empty routes.

Likely follow-up: What if you disagree with the reviewer?

Follow-up answer: Bring a small reproduction, requirements/source evidence and trade-offs; ask what failure mode they are targeting; agree on a measurable invariant. Stay focused on code behavior.

Common mistake: Framing review as a personal dispute.

Q47. What did you learn from this project?

Short answer: A working interface and a high synthetic score do not by themselves validate the underlying real-world model.

Strong answer framework: Use a truthful learning example: explain how you came to distinguish the software pipeline from its data evidence, or how tracing the capacity denominator changed your interpretation. State what you personally studied or implemented. Then name the next concrete improvement, such as observed-data validation or a post-rounding feasibility test.

Evidence: `DATA_AUDIT.md`; synthetic metric and optimizer code.

Likely follow-up: What is your next step?

Follow-up answer: Acquire approved observed station-hour data with provenance, define a time-based baseline evaluation, and validate before presenting real-world performance claims.

Common mistake: Turning a learning narrative into claims of operational impact.

11. Final rapid-fire review

• **Observed passenger data?** None in the repository; current targets are synthetic.

• **Database?** No; runtime reads local CSV/JSON and joblib artifacts.

• **ML serving retrains?** No; it loads saved artifacts.

• **Large ignored CSV needed to serve?** Not in the current inference code path; training pipeline does need it or regeneration.

• **Crowding percentage?** Demand divided by estimated hourly boarding-clearance capacity, not onboard occupancy.

• **Train-by-train collision avoidance?** None; no train movement simulation.

• **Travel time source?** Weighted topology graph with coordinate/fallback travel edges and fixed transfer penalty.

• **Tests?** No automated test suite tracked.

• **23:30 open?** No; end comparison is exclusive.

• **Can the headway exceed fleet cap after rounding?** Yes, current code can; explain the discovered risk and desired invariant.

• **Best next project upgrade?** Sourced passenger and operations data with real holdout validation, not a more complex model first.

How to practice

1. Answer the short version in 15-25 seconds. 2. Give the strong answer in about one minute, then stop and invite follow-ups. 3. Open the evidence paths and explain the function/data flow in your own words. 4. Separate what the code does from what you personally did. 5. If a number or provenance is unknown, say so and explain how you would verify it.